Project 11.1 — ClientLens-Multi-Agent-Customer-360-Team-Over-MCP-Services

A practice twin of Project 11 — MCPCrew. Same skills — a Supervisor-led team where each specialist owns its *own* MCP server connection, a Writer synthesizes one report, and a dead service degrades the team instead of crashing it. A **new, real-world job**. **There are no steps here on purpose.** Read the story, build it yourself, and only open the hints or the solution if you get stuck.

Type: Multi-agent (1 Supervisor + 3 specialists) · **Stack:** Python, LangGraph (`Command`), `mcp` SDK, OpenAI SDK, asyncio, Pydantic · **Level:** Advanced

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

A support rep opens a ticket and needs the whole picture of the customer *right now*: what have they bought, and what have they complained about? At most companies those two facts live in **two different systems** owned by two different teams — an orders service and a support-tickets service — each reachable only through its own connection. Pulling them together by hand, in two browser tabs, is exactly the busywork an assistant should do.

You are going to build **ClientLens**: a Supervisor-led team that assembles a **customer-360** summary from those separate services. Each is its own MCP server:

- An **Orders Agent** connects to the orders service and pulls the customer's recent orders.
- A **Tickets Agent** connects to the support-tickets service and pulls their open/closed tickets.
- A **Supervisor** decides, per request, who runs — and *only* routes; it never touches a connection itself.
- A **Writer** turns both specialists' findings into one coherent 360 summary, not two raw dumps.

And because production services go down, ClientLens **degrades gracefully**: if the tickets service is unreachable, the Tickets Agent catches that inside its own node, and the summary is built from orders alone — clearly saying the tickets service couldn't be reached, never pretending it was checked.

The one lesson of this project: a real "assistant team" doesn't keep all its tools in one codebase — different capabilities are owned by different services. Give each specialist its own connection, let one routing layer coordinate them, and make one dead service a partial-but-honest answer instead of a crash.

How it works, from a request to a 360 summary (picture the whole machine)

Read this once and the whole project should form in your head.

Everything runs on a LangGraph graph: `START → supervisor`, and the supervisor routes with `Command` to the Orders Agent, the Tickets Agent, or the Writer, coming back to the supervisor after each specialist. The specialist nodes are **async**, because each one opens its own MCP connection — the Orders Agent to the orders server, the Tickets Agent to the tickets server — discovers that server's tools, runs a small ReAct loop (pull the data), and writes its findings into shared state. The whole connection is opened and closed inside that one async call, so a specialist can run as a graph node without its teardown tangling with the event loop. The graph is run with `.ainvoke()`.

The Supervisor reads the shared state and picks who's next: orders not pulled yet → Orders Agent; tickets not pulled and not marked unavailable → Tickets Agent; otherwise → Writer. It never opens a connection — it only names the next node. The Writer is a plain LLM call that weaves both findings into one summary, and it reads a `tickets_unavailable` flag so it can say, plainly, when a source was missing.

The resilience lives **inside** the specialist: the connect-and-pull is wrapped in a timeout and a `try/except`, so a service that's down returns empty findings with `unavailable=True` instead of letting an exception escape and crash the graph. That flag threads through the Supervisor (which then skips the dead service) to the Writer (which names the gap).

That's the whole machine: **supervisor routes → each specialist opens its own MCP connection and pulls its data → the Writer synthesizes one 360 summary → a dead service becomes an honest partial summary, not a crash.**

What you are building (the pieces)

- Two MCP servers (orders, support-tickets) and two specialist agents that each own one connection.
- A shared `run_specialist` ReAct loop that opens/closes its connection inside one async call (with the degradation `try/except`).
- A `Command`-based Supervisor that routes to specialist nodes and reads results from shared state.
- A Writer that synthesizes one 360 summary and names any unavailable source.
- An async LangGraph graph run with `.ainvoke()`.

Inputs: a customer id and a request. **Outputs:** one 360 summary; or an honest partial summary when a service is down. **Rules:** the Supervisor only routes; each specialist owns its own connection; a dead service degrades the team, never crashes it.

A real example to build and test against

- `"Give me a customer-360 for c-42: orders and support tickets."` → Orders Agent + Tickets Agent + Writer → one summary (the laptop/dock orders *and* the open dock-charging ticket).
- `"What has c-42 ordered?"` → Orders Agent only.
- The 360 request with the **tickets service not started** → a summary from orders alone, clearly saying the tickets service couldn't be reached.

How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 8's MCP client and Project 4's Supervisor if they're hazy.)
2. Plan it on paper: which pieces you reuse from a single-connection MCP client, and which from a `Command` Supervisor — most of this is combining two patterns you've built before.
3. Build it yourself. A good order: each specialist standalone (own MCP connection) → the Supervisor routing between them → the Writer → graceful degradation. Your call.
4. Test it on the real example: a both-service 360, an orders-only request, and a tickets-down degraded summary.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

This is the real shape of an internal "assistant team": different capabilities owned by different services, each behind its own connection, pulled together by one routing layer. A customer-360 here, but the same shape powers an incident-response team (logs service + metrics service), a sales-brief team (CRM + news), an ops assistant (tickets + deploys). Two habits make it production-grade and are the whole point: keep each specialist's connection its own (so one failure can't touch another), and make a dead service a partial-but-honest answer instead of a crash. Build ClientLens and you can assemble any multi-service assistant team.

Done when

- A 360 request runs both specialists, each on its own MCP connection, and the Writer produces one synthesized summary.
- The Supervisor only routes — it never opens a connection.
- With the tickets service down, the Tickets Agent catches it inside its node; the summary is built from orders and names the gap; nothing crashes.
- Specialist nodes are `async` and the graph is run with `.ainvoke()`.

- Your tests pass: settings, the Supervisor routing (hand-made states, no model), and a live both-service + degradation run.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.